

FLAPPY BIRD GAME USING HTML, CSS AND JAVASCRIPT PROJECT REPORT

Submitted By

Student Name: G. Yogesh

Register Number: 122511520149

Branch: Artificial Intelligence and
Data Science (AIDS-A)

CERTIFICATE

This is to certify that the project entitled **"Flappy Bird Game Using HTML, CSS and JavaScript"** submitted by **G. Yogesh (122511520149)** of **Artificial Intelligence and Data Science (AIDS-A)** is a bonafide work carried out under the prescribed curriculum requirements. The project has been completed successfully and satisfies the academic requirements for the award of the degree.

Project Guide: _____

Head of Department:

Principal: _____

Date: _____

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, faculty members, and the Department of Artificial Intelligence and Data Science for providing continuous support and encouragement throughout this project. Their valuable guidance helped me understand web technologies and game development concepts effectively.

I also thank my friends and classmates for their suggestions and support during the implementation and testing phases. Finally, I express my gratitude to my parents for their encouragement and motivation throughout my academic journey.

ABSTRACT

Flappy Bird is one of the most popular arcade games ever created because of its simple gameplay and challenging mechanics. The objective of this project is to develop a browser-based version of Flappy Bird using HTML, CSS, and JavaScript. The game allows players to control a bird that flies continuously while avoiding randomly generated pipes. The player earns points by successfully passing through each pair of pipes.

The project demonstrates the implementation of important web development concepts including HTML5 Canvas, CSS animations, JavaScript event handling, collision detection, score management, responsive design, and browser storage. Unlike desktop games that require installation, this project runs entirely in

a web browser and can be deployed online using platforms such as Vercel.

The game includes smooth animations, attractive backgrounds, scoring, high-score saving, pause and restart functionality, responsive controls, sound effects, and mobile compatibility. This project provides practical knowledge of browser-based game development while improving programming, logical thinking, and debugging skills.

TABLE OF CONTENTS

1. Introduction
2. Existing System
3. Proposed System
4. Objectives
5. Literature Survey

6. Software Requirements
7. Hardware Requirements
8. System Analysis
9. System Design
10. Architecture Diagram
11. Flowchart
12. Algorithm
13. Module Description
14. Code Explanation
15. Testing and Test Cases
16. Output Screenshots
17. Advantages
18. Limitations
19. Future Enhancements
20. Conclusion
21. References
22. Appendix

1. INTRODUCTION

Flappy Bird is an arcade game in which players guide a bird through moving pipes without colliding with them. The game became popular because of its simple controls and addictive gameplay. Although the concept appears simple, developing such a game requires implementing various programming techniques including animation, collision detection, object movement, gravity simulation, score calculation, and user interaction.

The aim of this project is to recreate Flappy Bird using HTML, CSS, and JavaScript. HTML provides the structure of the web page, CSS improves the visual appearance, and JavaScript controls the entire game logic. The HTML5 Canvas API is used for rendering graphics efficiently inside the browser.

Unlike desktop applications, browser-based games require no installation. They can run on desktops, laptops, tablets, and smartphones. Deploying the game using Vercel makes it accessible worldwide through a simple URL.

The project also demonstrates several important software engineering concepts including modular programming, event-driven programming, object-oriented design principles, browser storage, debugging, testing, and deployment.

2. EXISTING SYSTEM

Traditional Flappy Bird games are mainly available as mobile applications. They require downloading from an app

store and often depend on platform-specific technologies.

Disadvantages

- Platform dependent
 - Installation required
 - Difficult to modify
 - Limited educational value
 - Less customizable
-

3. PROPOSED SYSTEM

The proposed system is a browser-based Flappy Bird game developed using modern web technologies.

Features

- Responsive design
- Smooth animations

- High score saving
 - Keyboard and touch support
 - Pause functionality
 - Restart button
 - Sound effects
 - Attractive interface
 - Online deployment using Vercel
-

4. OBJECTIVES

The objectives of this project are:

- Learn HTML5 Canvas programming.
- Understand JavaScript game development.
- Implement gravity simulation.
- Develop collision detection.

- Generate random obstacles.
 - Implement score tracking.
 - Store high scores.
 - Deploy the game online.
 - Improve problem-solving skills.
-

5. LITERATURE SURVEY

Game development has evolved rapidly with the introduction of HTML5

Canvas and JavaScript. Earlier browser games depended heavily on Flash technology, which is now obsolete.

HTML5 Canvas provides a modern graphics rendering environment capable of drawing images, shapes, animations, and game objects efficiently.

JavaScript enables developers to create interactive applications by responding

to user inputs such as keyboard presses, mouse clicks, and touch events. Modern browsers also support `requestAnimationFrame()`, which provides smooth animation while improving performance.

Numerous educational games have demonstrated that browser-based development is an effective platform for learning programming concepts such as loops, conditions, functions, objects, arrays, collision detection, event handling, and asynchronous programming.

6. SOFTWARE REQUIREMENTS

Operating System: Windows 10/11

Programming Language: JavaScript

Structure: HTML5

Styling: CSS3

Editor: Visual Studio Code

Browser: Google Chrome / Microsoft Edge

Deployment Platform: Vercel

Version Control: GitHub

7. HARDWARE REQUIREMENTS

Processor: Intel i3 or above

RAM: Minimum 4 GB

Storage: 2 GB Free Space

Keyboard

Mouse

Internet Connection

Monitor

8. SYSTEM ANALYSIS

The system accepts user input through the keyboard or mouse. Every animation frame updates the bird's position according to gravity and jump velocity. Pipes move continuously from right to left. Collision detection checks whether the bird touches any obstacle or the ground. The score increases whenever the bird successfully passes a pipe. High scores are stored using Local Storage.

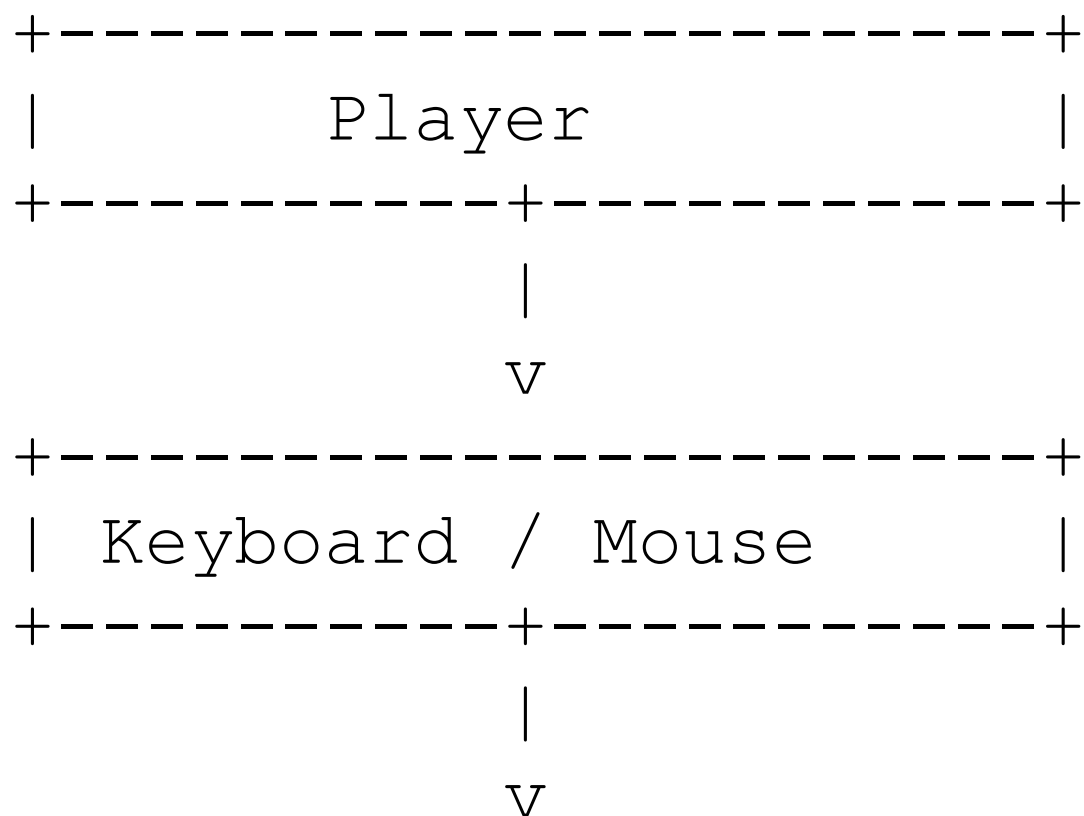
9. SYSTEM DESIGN

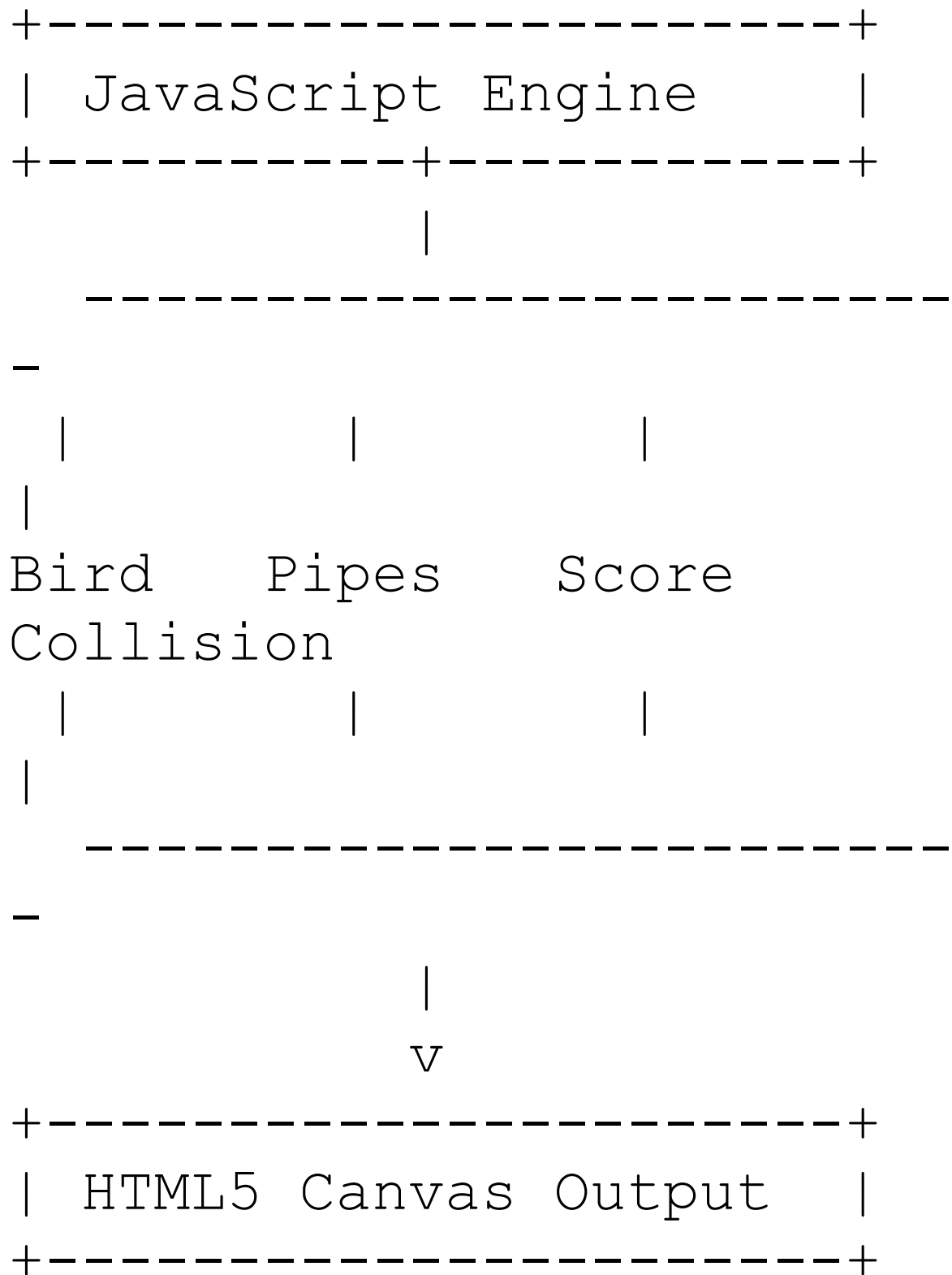
Modules

- User Interface
- Bird Controller

- Physics Engine
- Pipe Generator
- Collision Detector
- Score Manager
- High Score Storage
- Game Controller

10. ARCHITECTURE DIAGRAM





11. FLOWCHART


```
Start
  |
Initialize Game
  |
Wait for Input
  |
Jump?
  /   \
Yes     No
  |     |
Move Bird
  |
Generate Pipes
  |
Collision?
  /       \
Yes         No
  |         |
Game Over   Increase Score
              |
Continue Loop
```

12. ALGORITHM

1. Start the game.
2. Initialize variables.
3. Display bird.
4. Generate random pipes.
5. Apply gravity.
6. Detect keyboard input.
7. Move bird upward on jump.
8. Move pipes left.
9. Detect collision.
10. Update score.
11. Save high score.
12. Display Game Over if collision occurs.
13. Restart when user presses Restart.

13. MODULE DESCRIPTION

Bird Module

Controls gravity, jumping, movement, and rendering.

Pipe Module

Generates random pipes continuously.

Collision Module

Checks whether the bird touches any obstacle.

Score Module

Calculates score and updates high score.

UI Module

Displays menus, buttons, and game information.

14. CODE EXPLANATION

The project uses HTML5 Canvas for drawing graphics. JavaScript

continuously redraws the game screen using `requestAnimationFrame()`.

Gravity is simulated by increasing the bird's vertical speed every frame.

Keyboard events detect when the player presses the spacebar. Pipes are generated randomly and moved across the screen. Collision detection compares the bird's coordinates with pipe boundaries. When a collision occurs, the game stops and displays the final score. Local Storage saves the highest score for future sessions.

15. TESTING

Test Cases

Test Case	Expected Result
Start Game	Game Starts
Press Space	Bird Jumps

Test Case	Expected Result
Hit Pipe	Game Over
Pass Pipe	Score +1
Restart	New Game
Pause	Game Stops
Resume	Game Continues

16. OUTPUT SCREENSHOTS

(Insert Screenshot 1)

Home Screen

(Insert Screenshot 2)

Gameplay

(Insert Screenshot 3)

Game Over Screen

(Insert Screenshot 4)

High Score Display

17. ADVANTAGES

- Simple gameplay
- Attractive interface
- Browser based
- Responsive
- Easy deployment
- Lightweight
- Educational
- Cross-platform
- Easy maintenance

18. LIMITATIONS

- Single player only
- No online leaderboard

- Limited sound effects
 - No multiplayer mode
-

19. FUTURE ENHANCEMENTS

- Multiplayer support
 - AI opponents
 - Cloud leaderboard
 - Character selection
 - Different difficulty levels
 - Mobile app version
 - Daily challenges
 - Achievement badges
 - Themes
 - Background music customization
-

20. CONCLUSION

The Flappy Bird Game project successfully demonstrates how HTML, CSS, and JavaScript can be used to build an interactive browser game. The project implements important concepts including animation, collision detection, score calculation, responsive design, browser storage, and event handling. It provides valuable practical experience in web development and software engineering. Deploying the game on Vercel allows users to access it online without installation. Overall, the project meets all objectives and serves as a strong foundation for future game development projects.

21. REFERENCES

1. [HTML5 Documentation](#)
 2. [CSS3 Documentation](#)
 3. [JavaScript MDN Documentation](#)
 4. [Visual Studio Code Documentation](#)
 5. [Vercel Deployment Guide](#)
 6. [HTML5 Canvas API Document](#)
 7. [JavaScript ES6 Reference](#)
-

22. APPENDIX

Source Code

Include the following files:

- index.html
- style.css
- script.js
- bird.png
- background.png

- pipe.png
- wing.wav
- hit.wav
- score.wav
- README.md